Empirical Dependability Measurement of Apache Cassandra under Fault Injection: the Impact of Consistency Levels

Caio Farias Cavalcanti[1] and Gustavo Rau de Almeida Callou[1]

Federal Rural University of Pernambuco (UFRPE), Recife–PE, Brazil
{caio.farias,gustavo.callou}@ufrpe.br

Abstract. Distributed database systems such as Apache Cassandra offer tunable consistency levels that regulate the trade-off between availability and consistency. Well understood in theory, this trade-off nonetheless lacks empirical measurements of how different consistency levels affect service dependability under real faults. This work presents a measurement study that injects five fault types (replica crash, coordinator crash, network delay, packet loss, and network partition) into Cassandra under three consistency levels (ONE, QUORUM, ALL) and three load levels, with three repetitions each (135 runs). The results quantify two distinct phenomena: faults that cause unavailability (crash and partition), where ONE and QUORUM completely mask the fault while ALL exhibits an error rate of up to 25%; and timing faults (delay and loss), where throughput degrades by ∼98% regardless of the consistency level. We further measure the time to repair (MTTR) per fault type, from ∼15 s (crash) to ∼32 s (partition). The measurements extend prior work by introducing partition scenarios not previously studied and by providing empirical observed service availability and MTTR values.

Keywords: Dependability · Apache Cassandra · Fault injection · Consistency levels · Availability · Fault tolerance.

1 Introduction

Distributed NoSQL databases have become core infrastructure for large-scale applications, offering horizontal scalability and fault tolerance through replication. Apache Cassandra is a prominent representative of this category, adopted by organizations requiring high availability over large data volumes [1]. A defining feature of Cassandra is *tunable consistency*: for each operation, the application chooses how many replicas must participate, explicitly regulating the consistency–availability trade-off predicted by the CAP theorem [2, 3].

This trade-off is well understood at a theoretical level. What remains less explored is the *empirical quantification* of how the choice of consistency level affects the dependability of the service delivered to the client when the system faces real faults. Knowing that ALL "prioritizes consistency" does not tell *how much* availability is lost under a concrete fault, nor *how quickly* the service

recovers. These are dependability measures [4, 5] that operators need to make informed configuration decisions.

The object of study is the consistency level, and Cassandra suits it because the level is chosen per operation: ONE, QUORUM and ALL can be compared on the *same* cluster, data and fault. In consensus-based systems such as etcd (Raft), the guarantee is largely fixed by the protocol. The injection method (`systemctl`, `tc/netem`, `iptables`, client-side time series) does not depend on Cassandra; whether the trade-offs generalize is left to future work.

This work addresses that gap through a controlled measurement study: a five-node Cassandra cluster in the cloud is subjected to five fault classes under three consistency levels and three loads, and the throughput, latency and error rate are recorded before, during and after each fault.

The contributions are: (i) a reproducible fault-injection methodology for Cassandra, covering crash, timing, omission and partition faults with precise timeline control, whose scripts and data are publicly available [14]; (ii) the empirical quantification of the impact of the consistency level on dependability, distinguishing availability faults from timing faults; and (iii) measured availability and MTTR values, including partition scenarios not addressed in related work. The scope is restricted to measurement: this is the experimental stage of an undergraduate project, with analytical modeling as the subsequent stage.

The study is guided by three research questions:

- **RQ1:** How does the Cassandra consistency level affect service availability under crash and network partition faults?
- **RQ2:** How does the consistency level affect performance degradation under network timing faults (delay and packet loss)?
- **RQ3:** How do different fault types affect Cassandra recovery time (MTTR)?

Section 2 reviews related work and Section 3 the concepts used. Section 4 describes the experimental design and Section 5 the apparatus. Section 6 presents and discusses the results, and Section 7 concludes.

2 Related Work

Dependability evaluation of Cassandra has been approached through measurement and through analytical modeling. Ferreira et al. [6] experimentally evaluated Cassandra's performance under different consistency levels and loads, establishing a baseline of throughput and latency under normal operation. That work, however, focuses on the fault-free regime and does not characterize system behavior under node unavailability or network degradation. The present study adopts compatible load parameters for comparability, but extends the evaluation with a systematic fault-injection phase.

Other empirical studies cover complementary dimensions. Rosselli et al. [10] evaluated Cassandra's availability and fault tolerance by measuring the impact of a node failure on throughput and latency, without systematically varying the consistency level or covering network faults such as delay, loss and partition. Rahman et al. [11] proposed a framework, integrated with YCSB, for empirically

measuring consistency anomalies in key-value stores, a perspective orthogonal to this work, which centers on availability and recovery. In failure-oriented testing, the Jepsen project [12] became a reference by subjecting distributed databases, including Cassandra, to partitions and crashes in order to verify *correctness violations* of consistency guarantees; our work differs in goal and method, quantifying dependability metrics (error rate, throughput degradation and MTTR) as a function of the consistency level, under a controlled fault timeline.

Gomes et al. [9] proposed an analytical dependability model for Cassandra based on generalized stochastic Petri nets (GSPN), estimating availability from MTTF and MTTR parameters adopted hypothetically (MTTF of 2880 h and 5760 h; MTTR of 0.5 h and 1 h), in the absence of direct measurements. Since the quality of an analytical model depends on its input parameters, this work contributes *empirically measured* MTTR values per fault type.

Fault injection is a well-established *fault forecasting* technique for the experimental evaluation of dependability [4, 7, 8]. Network partitions, which materialize the CAP trade-off, have not, to our knowledge, been characterized as a dependability measure in the related Cassandra studies.

3 Background

3.1 Dependability and the fault–error–failure chain

Dependability is the ability of a system to deliver a service that can justifiably be trusted [4], encompassing attributes such as availability (readiness for correct service), reliability (continuity of correct service) and maintainability (ability to undergo repair). The Avižienis et al. framework structures dependability threats along the causal chain *fault* $\rightarrow$ *error* $\rightarrow$ *failure*: a **fault** is the hypothesized or actual cause of an error (e.g., a process terminating); an **error** is the part of the system state that may lead to a failure (e.g., an unreachable replica); and a **failure** occurs when the delivered service deviates from the correct one (e.g., an operation returning an error to the client). Between error and failure act the *fault tolerance mechanisms*: when a mechanism prevents an error from propagating to the service, the fault is said to be *masked*; controlled fault injection measures whether each fault crosses these defenses.

3.2 Cassandra's replication and consistency levels

Cassandra partitions data over a *token* ring and replicates each partition across a number of nodes given by the *replication factor* (RF). With $RF = 3$, each data item exists on three distinct nodes, which constitutes its fundamental fault tolerance mechanism.

The *consistency level* (CL) determines, per operation, how many replicas must respond for it to be considered successful: **ONE** requires a single replica (tolerates the loss of up to $RF - 1$ replicas); **QUORUM** requires a majority, $\lfloor RF/2 \rfloor + 1$ replicas (two, for $RF = 3$; tolerates the loss of one); and **ALL** requires all RF replicas (tolerates no loss).

A consistency level is a per-operation parameter, not a consistency *model*; the model it yields depends on the levels used for reads (R replicas) and writes (W). With ONE, a read may return stale data, and the system offers only *eventual consistency*. When $R+W > RF$, as with QUORUM for both or ALL, every read set intersects the replicas that acknowledged the latest completed write. This is still weaker than *linearizability*, which in Cassandra requires *lightweight transactions* (Paxos), not used here. We measure the availability and performance *cost* of each level, not the consistency obtained, which is the subject of [11, 12].

The CL is thus an application-level mechanism for controlling the behavior observed during node and network failures: ONE favors availability (responding with any reachable replica), while ALL refuses to respond without all of them. An important aspect for interpreting the results is that, in Cassandra, *every write is sent to all replicas* regardless of the CL, which defines only how many acknowledgments the coordinator waits for.

3.3 Measurable dependability attributes

From observing the service under faults we derive the measures of interest, drawing on two complementary traditions: the attributes of the taxonomy of Avižienis et al. [4] and Meyer's *performability* [5]. **Availability** (readiness for correct service) is measured at the operational level, by the fraction of operations served correctly, $A = 1 - (\text{errors/total operations})$; we call this *observed service availability*. It departs from the classical steady-state availability $A = \text{MTTF}/(\text{MTTF} + \text{MTTR})$ for two reasons: faults are injected at fixed instants, so there is no MTTF to measure; and in a request-serving system each operation is a demand for service, so the fraction served correctly is the availability the client experiences, the request-based form used in service-level objectives. Since it depends on the observation interval, we report it over the whole run and inside the fault window. **Performability** [5] is the degree of throughput and latency degradation during the fault, when the service remains available but with reduced quality; it is not an attribute of the Avižienis et al. taxonomy, but a composite measure of performance and dependability suited to degradable systems. **Maintainability** is assessed through the recovery time (MTTR) after the injected fault.

4 Methodology

4.1 Experimental design

The study is organized in two phases. The *baseline phase* measures throughput and latency under normal operation, varying the consistency level (ONE, QUORUM, ALL) and the load (10, 30 and 50 client threads), with three repetitions per cell. The *fault-injection phase* adds the fault-type dimension, totaling $5 \times 3 \times 3 \times 3 = 135$ controlled runs.

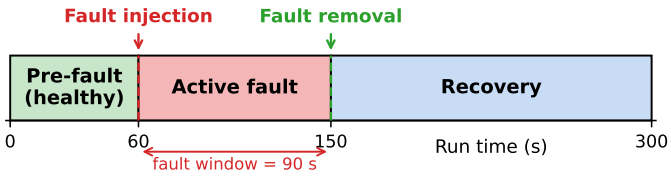


Fig. 1. Controlled timeline of a fault-injection run.

4.2 Injected fault types

Five fault types cover the main classes of the dependability taxonomy. **Replica crash** (`node_crash`): termination of the Cassandra process on a replica node, via `systemctl stop`, which tests masking by replication. This is an *orderly* shutdown, in which the node announces its departure to its peers; we chose it to isolate replica loss from failure-detection delay and to restart cleanly in a long automated battery (abrupt termination is discussed in the threats to validity). **Coordinator crash** (`coord_crash`): termination of the process on the client's contact node, testing the *driver*'s routing failover. **Network delay** (`net_delay`): insertion of 200 ms ± 50 ms of latency into a node's egress traffic, via `tc/netem` (a timing fault). **Packet loss** (`net_loss`): dropping 25% of a node's egress packets, also via `tc/netem` (an omission fault). **Network partition** (`net_partition`): isolation of one node from the others, via `iptables` rules, keeping the node alive but unable to communicate; it materializes the CAP theorem scenario.

4.3 Per-run timeline

Each run lasts 300 s and follows a controlled timeline (Fig. 1): 60 s healthy; fault injection at $t = 60$ s; the fault held for 90 s; removal at $t = 150$ s; and recovery until $t = 300$ s. The 90 s window exceeds the fault detection time of Cassandra's *gossip* protocol (on the order of 10 s), ensuring an effectively-detected fault window long enough for measurement.

4.4 Workload and metrics

The load is generated by YCSB (Yahoo! Cloud Serving Benchmark) [13] with *Workload A* (50% reads, 50% updates) over 100,000 records. Each run is bounded by wall-clock time (300 s) rather than by operation count, ensuring the fault window occurs within the measurement period. YCSB records every 10 s the throughput, latency and counts of successful and failed operations, constituting the time series from which we derive the metrics; the actual injection, removal and recovery instants are recorded by the injection harness, allowing the series to be aligned to the pre-fault, during-fault and post-fault windows.

From these series we compute the **error rate** (errors over total operations) and the **observed service availability** (1− error rate), used for crash and partition; the **throughput degradation** ($1 - \text{throughput}_{\text{during}}/\text{throughput}_{\text{pre}}$),

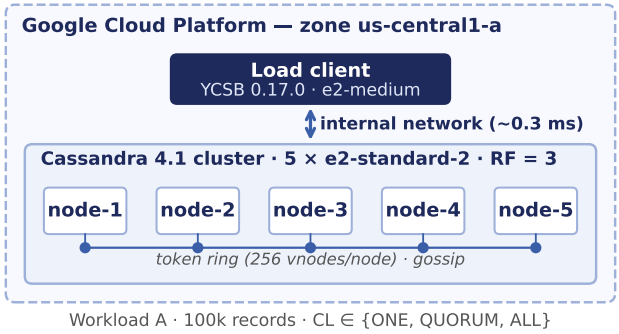


Fig. 2. Architecture of the experimental environment: a Cassandra 4.1 cluster of five `e2-standard-2` nodes (RF=3, `SimpleStrategy`) and the YCSB load client (`e2-medium`), co-located in the `us-central1-a` zone of the Google Cloud Platform.

used for delay and loss; and the **MTTR**, the interval between fault removal and the confirmation, via `nodetool status`, that the cluster returned to a five-node-up state. By construction, the MTTR is only meaningful for faults that bring the node down from *gossip* (crash and partition); for timing faults the relevant recovery is that of throughput, observable in the post-fault window.

5 Environment and Experimental Apparatus

5.1 Infrastructure

The experiment ran on the Google Cloud Platform, in the `us-central1-a` zone (Fig. 2). The Cassandra cluster comprises five `e2-standard-2` virtual machines (2 vCPUs, 8 GB RAM each) running Cassandra 4.1 on Ubuntu 22.04, and load is generated by a dedicated VM (`e2-medium`, 2 vCPUs, 4 GB) co-located in the same zone so that client–cluster latency (∼0.3 ms on the internal network) is not a confounding factor.

5.2 Cassandra configuration

The cluster uses $RF = 3$ with `SimpleStrategy` replication (suitable for a single datacenter), `num_tokens`=256, `Murmur3Partitioner` and the `GossipingPropertyFileSnitch`, with two *seed* nodes. The resulting token distribution assigns each node ∼58–62% of the key space in effective ownership, which is relevant for interpreting the results under CL=ALL.

5.3 Load and tooling

Load generation uses YCSB 0.17.0 [13]. Fault injection is performed by scripts that orchestrate, synchronized with the YCSB run, process termination and

Table 1. Experiment parameters.

Parameter	Value	Parameter	Value
Cassandra nodes	5 × e2-standard-2	Load (threads)	10, 30, 50
Load client	1 × e2-medium	Repetitions per cell	3
Cassandra version	4.1	Workload	YCSB A (50/50 read/update)
Replication factor	3 (SimpleStrategy)	Records	100,000
Consistency levels	ONE, QUORUM, ALL	Run duration	300 s
Total runs (faults)	135	Fault window	90 s ($t = 60$–150 s)

restart (`systemctl`) for the crash scenarios, queueing discipline manipulation (`tc/netem`) for delay and loss, and firewall rules (`iptables`) for partition. Post-removal verification and idempotent cleanup prevent residual fault state from contaminating subsequent runs, a critical requirement in a long battery of automated runs. Table 1 summarizes the core experimental parameters. The fault-injection and analysis scripts, the raw logs and the per-run data are publicly available [14].

6 Case Study: Results and Discussion

In the fault-free baseline, the coefficient of variation across repetitions stayed below 3%. The results are grouped by the predominant attribute: availability (crash, partition) and performability (timing faults).

6.1 Availability under crash and partition

Table 2 presents the mean error rate per fault type and consistency level. The pattern is clear and consistent with quorum theory: under replica or coordinator crash, ONE and QUORUM maintain a near-zero error rate (zero failed operations in every cell, except for a single one in 2.6×10^7 under ONE for the coordinator crash), since with $RF = 3$ the loss of one replica leaves two available, sufficient for ONE (one) and QUORUM (two). The fault is therefore *masked* by replication, and both deliver ∼100% observed service availability. Under ALL, the requirement of all three replicas causes every operation whose replica set includes the unavailable node to fail, resulting in a mean error rate of ∼24–25% over the entire run. This is lower than the fraction of affected replicas because it is diluted by the healthy periods.

The dilution is not proportional to time (90 of 300 s, 30%). Throughput after the fault (∼4400–5700 ops/s) is lower than before it (∼7100–8200 ops/s), so the fault window holds 33–41% of the run's operations. We attribute the lower post-fault throughput to the reintegration of the node, which under ALL takes part in every operation on its replicas; this explanation is inferred from Cassandra's behavior, not instrumented.

Measured *inside the fault window*, from the 10 s YCSB samples, the error rate under ALL is 56.7% for replica crash, 55.6% for coordinator crash and 56.7% for partition (standard deviation below 0.1 points). Replica crash and partition

Table 2. Error rate (%) per fault type and consistency level (mean ± standard deviation over loads and repetitions; $n = 9$ per cell: 3 loads × 3 repetitions). In parentheses, the absolute number of failed operations summed over the nine runs; each cell totals $1.6–2.8 \times 10^7$ operations.

Fault type	ONE	QUORUM	ALL
Replica crash	0.00±0.00 (0)	0.00±0.00 (0)	24.29±4.24 (3,902,961)
Coordinator crash	0.00±0.00 (1)	0.00±0.00 (0)	25.25±1.45 (4,008,616)
Network partition	0.05±0.05 (16,305)	0.05±0.10 (9,768)	19.01±1.74 (3,442,518)

target the *same* node and agree to the third decimal (56.65% and 56.66%): the fault type changes for how long operations fail, not which ones. All three are close to the 60% expected from $RF = 3$ over five nodes under uniform access; we attribute the gap to the *zipfian* key distribution of *Workload A* [13]. Part of the errors occurs *after* removal: service returns ∼13–15 s after a crash is removed and ∼3 s after the partition is removed. The mean rate (Table 2) and the instantaneous one are complementary: the former quantifies the amortized cost over the entire run, the latter isolates the severity perceived while the node is in fact unavailable.

An original result lies in the comparison between crash and partition under ALL: the partition produces a *lower* mean error rate (∼19%) than the crash (∼24–25%), even though the instantaneous severity measured on the *same* target node is identical (∼56.7%). The difference does not lie in what happens *during* the fault, since under ALL an isolated but live node is as unreachable as a dead one, but in the *duration* of the service impairment: ∼98 s under partition against ∼124–129 s under the crashes. Removing an `iptables` rule restores connectivity at once, whereas a restarted process must first rejoin the cluster. This dissociation between *fault removal* and *service restoration* had not been characterized in related work. Figure 3 (center) also shows a brief QUORUM error spike (∼18%) in the first ∼10 s of the partition, invisible in aggregate metrics, which we attribute to *gossip* convergence.

6.2 Recoverability: time to repair

Table 3 presents the MTTR per fault type (RQ3). The crashes recover in ∼15–19 s and the partition in ∼32 s, which we attribute to the reconvergence of *gossip* views on both sides. This control-plane MTTR differs from the recovery perceived by the client: under partition, service returns ∼3 s after removal, well before *gossip* converges, whereas under the crashes the two practically coincide (∼13–15 s). Models parameterized by MTTR should state which recovery represents the downtime.

6.3 Performability under delay and loss

Table 4 presents the throughput degradation under timing faults. The degradation is severe (∼97–98%) and *nearly independent of the consistency level*: the

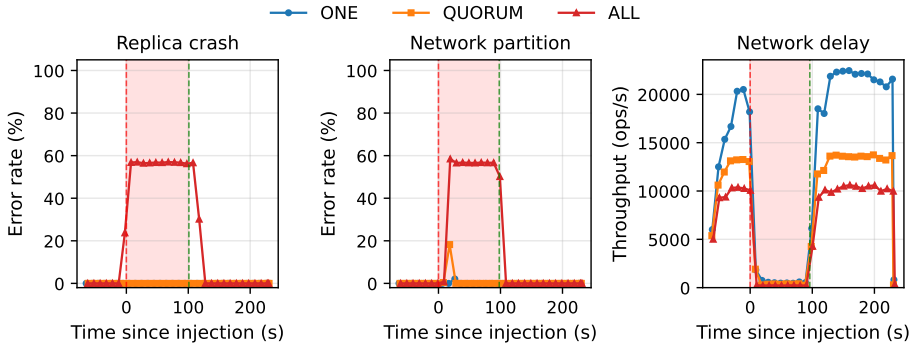


Fig. 3. Time series at 50 threads (median repetition), with time realigned to the fault injection instant ($t = 0$) and the fault window shaded. Left and center: error rate under replica crash and partition: only ALL reacts, with ONE and QUORUM at ∼0%. Right: throughput under network delay: all three levels degrade jointly.

Table 3. Time to repair (s): from fault removal until `nodetool status` shows five nodes up (control plane). Mean ± standard deviation; $n = 27$ per fault type (3 levels × 3 loads × 3 repetitions).

Fault type	Mean ± SD	Min.	Max.
Replica crash	15.19±2.76	11	25
Coordinator crash	19.15±7.27	12	34
Network partition	31.93±7.90	23	47

differences between levels are of the order of the standard deviation (0.2–0.6 points). The error rate remains zero, so this is a timing failure rather than an omission. We attribute the CL independence to the write path: every write is sent to all replicas, so a slow node delays the operations involving it whatever the number of acknowledgments awaited.

The whole-run p99 (9–22 ms) hides this, as the slow operations are ∼1% of the total. Per 10 s window, the p99 rises from ∼11 ms to a median of ∼920 ms during the fault, and loss is worse than delay (∼1.2–2.3 s against ∼0.6–0.9 s) despite equal throughput degradation.

6.4 Synthesis

The consistency level controls the trade-off *under availability faults* (RQ1) but is *ineffective under timing faults* (RQ2): it protects against an unavailable node, not against network degradation.

6.5 Threats to validity

Six threats should be noted. The first concerns generalization: each fault type was injected on a *single, fixed target node*. This deliberate choice isolates the

Table 4. Throughput degradation (%) under timing faults (mean ± standard deviation over loads and repetitions; $n = 9$ per cell).

Fault type	ONE	QUORUM	ALL
Network delay	97.54±0.45	97.60±0.44	98.35±0.61
Packet loss	97.22±0.53	96.70±0.22	98.28±0.28

fault type as the only varying factor between crash and partition, but makes the absolute severity of ∼56.7% specific to that node's token range and to the hot-key set of the *zipfian* generator. The same dependence explains the higher variance observed under ALL at high load. Second, three repetitions per configuration ($n = 9$ per fault and level, $n = 27$ per fault for the MTTR) make the dispersion estimates imprecise. Our claims rest on contrasts far larger than this dispersion: zero versus 19–25% errors, and 97–98% degradation at every level. Differences of the order of one standard deviation are not claimed, for instance between the degradation of each level under timing faults, or between the MTTR of replica and coordinator crash. The MTTR is polled over SSH, which limits its resolution to ∼10 s. Third, the crashes are orderly shutdowns. A later check on a new cluster (30 threads, four repetitions) found zero failed operations under ONE and QUORUM also with `SIGKILL` and with an emulated host failure; but under ALL service took ∼48 s to return after `SIGKILL` (vs. ∼18 s after an orderly stop), and a host failure caused ∼20 s in which over 1% of operations took more than 200 ms under ONE and QUORUM. Our crash recovery times are thus lower bounds for abrupt failures. Fourth, fixed-instant fault injection does not measure the MTTF, only the MTTR. Fifth, the client used all five nodes as contact points under the default load-balancing policy; latency-aware routing could attenuate the degradation under timing faults. Finally, one partition run exhibited anomalous behavior from transient cluster instability, and was re-executed after a health check.

7 Conclusion

This work measured the impact of Cassandra's consistency levels on service availability, performance and recovery under controlled fault injection. Over 135 runs, crash and partition are masked by ONE and QUORUM but cause ∼20–25% errors under ALL, while timing faults degrade throughput by ∼98% at every level. The MTTR ranges from ∼15 s (crash) to ∼32 s (partition), and service restoration is dissociated from fault removal: ∼3 s after a partition, but only once a crashed node rejoins.

This paper reports the experimental stage of an ongoing undergraduate research project. As future work, we plan to: increase the number of repetitions per cell; inject abrupt crashes and multiple concurrent or correlated faults; investigate additional workloads and replication factors; apply the method to other systems, such as Raft-based stores like etcd, to test whether the trade-offs generalize; build an analytical dependability model (stochastic Petri nets or relia-

bility block diagrams) using the MTTR measured here as an input parameter, enabling cross-validation between the measured and model-predicted availability rates; and perform sensitivity analysis of parameters not directly measurable through fault injection, such as the MTTF.

References

1. Lakshman, A., Malik, P.: Cassandra: a decentralized structured storage system. ACM SIGOPS Operating Systems Review **44**(2), 35–40 (2010)
2. Brewer, E.A.: Towards robust distributed systems. In: Proc. ACM PODC, pp. 7–10 (2000)
3. Gilbert, S., Lynch, N.: Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News **33**(2), 51–59 (2002)
4. Avižienis, A., Laprie, J.C., Randell, B., Landwehr, C.: Basic concepts and taxonomy of dependable and secure computing. IEEE Trans. Dependable and Secure Computing **1**(1), 11–33 (2004)
5. Meyer, J.F.: On evaluating the performability of degradable computing systems. IEEE Trans. Computers **C-29**(8), 720–731 (1980). https://doi.org/10.1109/TC.1980.1675654
6. Ferreira, S., Andrade, E., Mendonça, J.: Uma abordagem experimental para avaliar os níveis de consistência do banco de dados NoSQL Cassandra [An experimental approach to evaluate the consistency levels of the Cassandra NoSQL database]. In: Anais do XXII Simpósio em Sistemas Computacionais de Alto Desempenho (WSCAD), pp. 156–167. SBC, Porto Alegre (2021). https://doi.org/10.5753/wscad.2021.18520
7. Borges, I., Andrade, E., Silva, F.A., Callou, G.: Availability evaluation of a video surveillance system with distributed storage. Cluster Computing (2025). https://doi.org/10.1007/s10586-024-04962-x
8. Silva, A., Callou, G.: Models for availability evaluation of file servers in private clouds. Computing (2025). https://doi.org/10.1007/s00607-024-01372-0
9. Gomes, C., Borba, E., Tavares, E., Nogueira de O. Junior, M.: Performability model for assessing NoSQL DBMS consistency. In: Proc. 2019 IEEE International Systems Conference (SysCon), pp. 1–6. IEEE (2019). https://doi.org/10.1109/SYSCON.2019.8836757
10. Rosselli, M., Niemann, R., Ivanov, T., Tolle, K., Zicari, R.V.: Benchmarking the availability and fault tolerance of Cassandra. In: Big Data Benchmarking (WBDB 2015). LNCS, vol. 10044, pp. 87–95. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-49748-8_5
11. Rahman, M.R., Golab, W., AuYoung, A., Keeton, K., Wylie, J.J.: Toward a principled framework for benchmarking consistency. In: Proc. 8th USENIX Workshop on Hot Topics in System Dependability (HotDep). USENIX Association (2012)
12. Kingsbury, K.: Jepsen: Cassandra. Jepsen analysis report (2013). `https://aphyr.com/posts/294-jepsen-cassandra`, accessed 17 Sep 2026
13. Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R.: Benchmarking cloud serving systems with YCSB. In: Proc. ACM SoCC, pp. 143–154 (2010)
14. Cavalcanti, C.F., Callou, G.R.A.: Fault-injection scripts and data for "Empirical Dependability Measurement of Apache Cassandra under Fault Injection" (2026). `https://github.com/caaiofarias/cassandra-fault-injection-ladc2026`